

Flutter 3: Visualizador de recetas

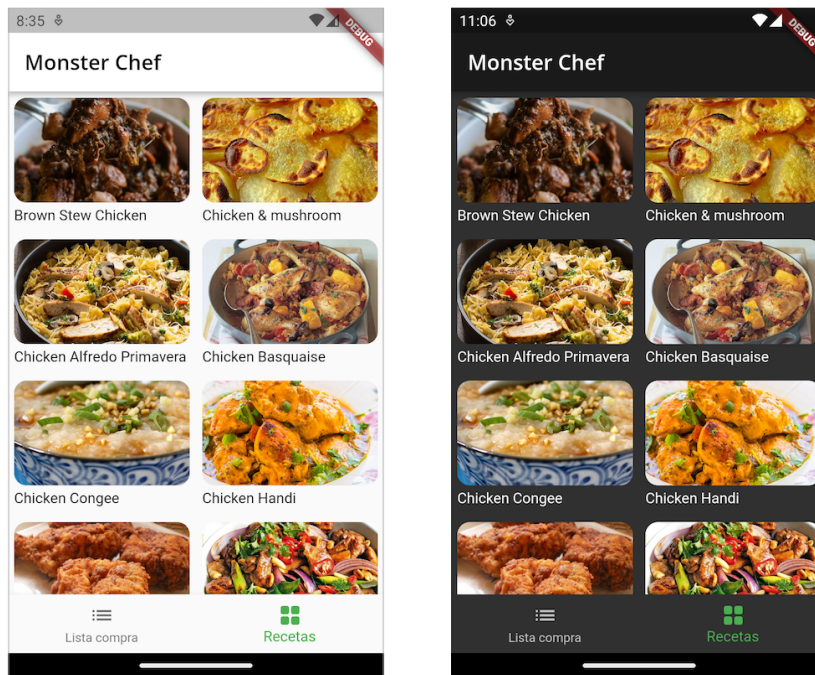
Contenidos

- Flutter 3: Visualizador de recetas
 - ¿Qué voy a hacer en esta sesión?
 - Pasos previos
 - 1 – Reorganizar los archivos de la aplicación en paquetes
 - 2 – Servidor de recetas
 - El servicio REST TheMealDB
 - Formato JSON de TheMealDB
 - Clases del modelo
 - Decodificar recetas en formato JSON
 - 3 – Leer las recetas de forma asíncrona
 - ¿Qué son y cómo se usan los `Future` ?
 - Leer las recetas usando `Future`
 - 4 – Construir la interfaz de forma asíncrona
 - 5 – Miniatura para mostrar las recetas
 - 6 – Usando el patrón galería
 - 7 – Adaptar la interfaz a los cambios de orientación
 - Mini ejercicios

¿Qué voy a hacer en esta sesión?

En esta sesión terminaremos la aplicación que comenzamos en la sesión anterior. Añadiremos la categoría de visualización de recetas. Para leer las recetas usaremos un servidor REST externo real que nos servirá los resultados en formato JSON. Aprenderemos a leerlos de forma asíncrona y a decodificarlos.

El aspecto de la aplicación terminada es este:



Pantalla de recetas terminada.

Mientras la escribimos paso a paso aprenderemos:

- A reunir algunos archivos del proyecto, relativos a algún modulo del programa, dentro de un `package` , para simplificar la forma en que se importan a otros archivos de paquetes diferentes.
- A conectar desde la aplicación móvil con un **servicio REST real** del que se leerán las recetas a través de internet. Los distintos componentes de la interfaz de usuario se construirán a medida que se vaya recibiendo desde el servidor, la información que muestran. Todo ello sin que se bloquee en ningún caso la hebra principal de la aplicación.
- A decodificar desde la aplicación Flutter, la información de recetas en formato JSON que nos proporciona el servidor.
- A adaptar la interfaz en función de su orientación (horizontal o vertical) de forma que la información mostrada se vea de la mejor forma posible.
- A utilizar entre otros los widgets de Flutter: `CircularProgressIndicator` , `FutureBuilder` , `ClipRRect` , `Image` , `GridView` y `OrientationBuilder` .

Pasos previos

Continuaremos con la aplicación tal y como se quedó en la sesión anterior. Seguiremos añadiendo funcionalidad a la que ya tiene.

Consejo: Antes de seguir, es una buena idea hacer un archivo comprimido en formato zip del contenido actual de la carpeta `flutter_session_2_3` por si en algún momento interesa recuperar una versión anterior.

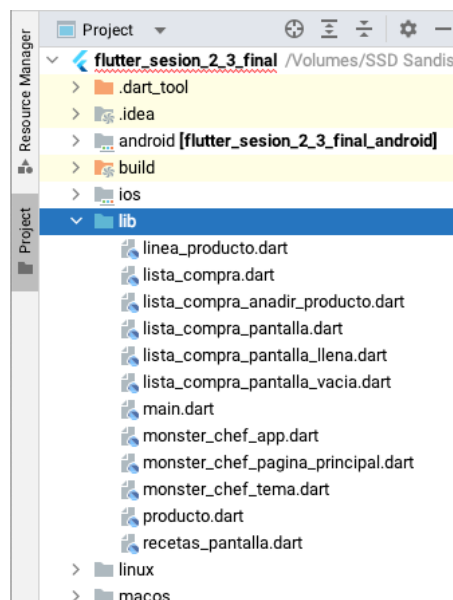
En este momento, la carpeta del proyecto es muy pesada ya que contiene además del código fuente, resultado de compilaciones, la apk de la aplicación, etc. Es una buena idea limpiar la carpeta de todo eso. Para eso se usa el menú `Tools – Flutter – Flutter Clean` de Android Studio.

`Flutter Clean` borra también las dependencias, así que recuerda que para volver a usar el proyecto de la carpeta comprimida, debes ejecutar en primer lugar `flutter pub get` para volver a reconstruirlas.

Consejo: Si estás usando un sistema de control de versiones como git, es el momento de construir una nueva rama (branch) para el proyecto.

1 – Reorganizar los archivos de la aplicación en paquetes

A medida que se añaden nuevas clases a la aplicación, aumenta el número de archivos en el proyecto, lo que puede hacer que se vuelva difícil de gestionar. En este momento, Monster Chef tiene todos estos archivos:



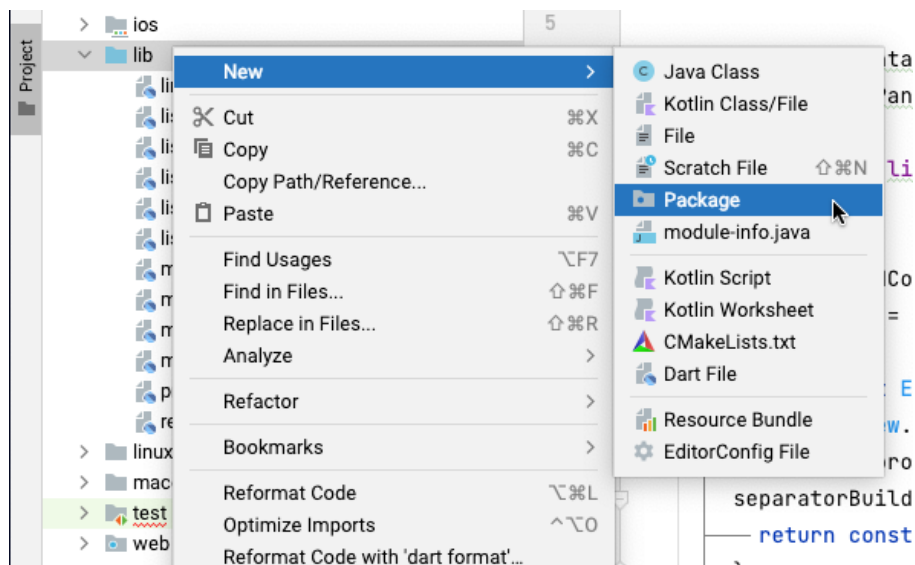
Archivos de código de la aplicación Monster Chef.

Dart ofrece la posibilidad de reunir archivos relacionados en `packages` y facilitar la forma en que se importan desde otros archivos situados en paquetes diferentes.

Vamos a añadir tres paquetes al proyecto:

- **Modelo:** En este paquete colocaremos las clases que representan el modelo de datos de la aplicación, como `Producto` que contiene un producto individual de la lista de la compra o `ListaCompra` que encapsula la lista de la compra completa.
- **Pantallas:** En este paquete colocaremos las clases que contienen las distintas pantalla de la aplicación como por ejemplo: `ListaCompraPantalla` o `MonsterChefPaginaPrincipal`
- **Componentes:** En este paquete colocaremos las `Widget` que contienen componentes a partir de los que se construyen las pantallas de la aplicación. Por ejemplo, aquí estará la clase `LineaProducto` que se usa en la pantalla `ListaCompraPantallaLlena` y que contiene las distintas vistas que representan toda la información sobre un producto de la lista de la compra.

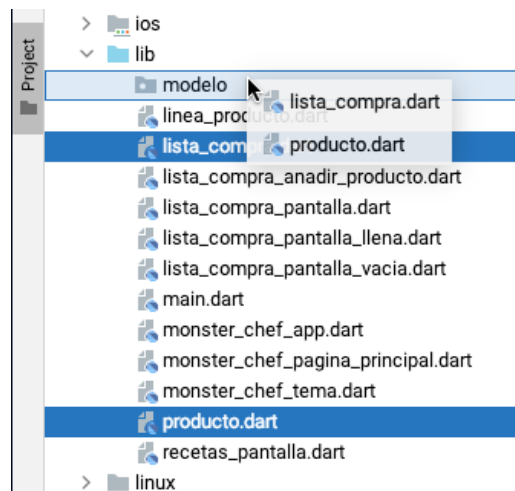
Para construir un paquete seguiremos un proceso similar al que se sigue para añadir nuevos archivos `.dart` al proyecto. Haremos clic-derecho sobre la carpeta `lib` y en el menú de contexto que aparece elegiremos la opción `New > Package`



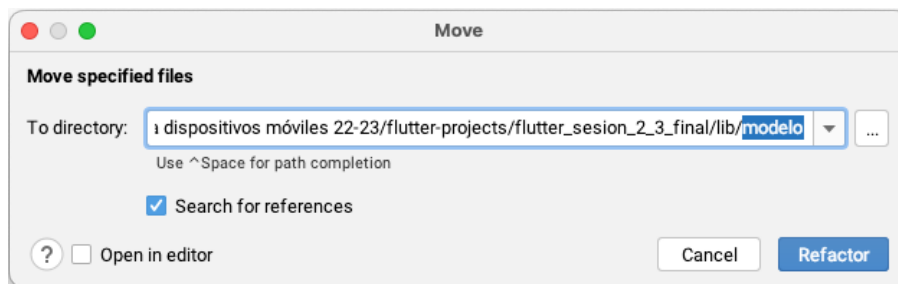
Aparece un pequeño cuadro de diálogo. Ahí hay que escribir el nombre del nuevo paquete, por ejemplo `modelo`



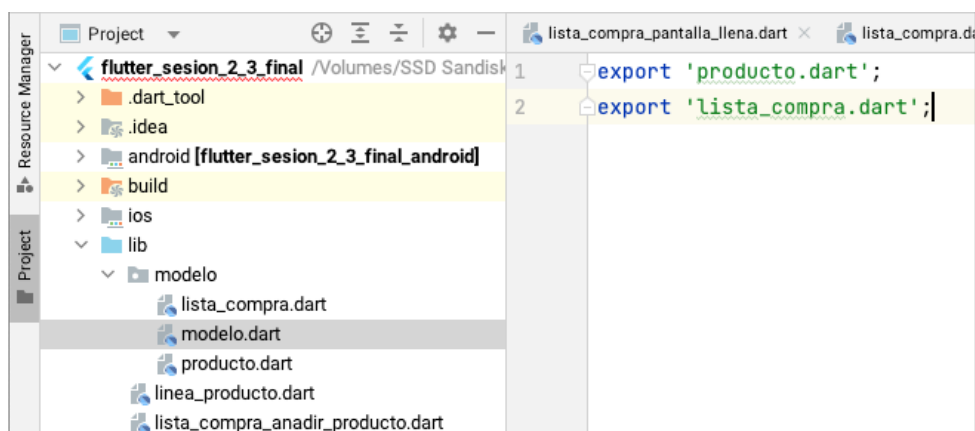
El nuevo paquete se muestra dentro de la carpeta `lib`. Para colocar uno o varios archivos dentro del paquete, solo hay que seleccionarlos y arrastrarlos sobre el icono del paquete:



y hacer clic en el botón **Refactor** del cuadro de diálogo que aparece:



Para facilitar la importación de los elementos de los archivos de un paquete se puede construir dentro del paquete un **barrel** que es un archivo `.dart` que contiene `export` para todos los archivos del paquete. De esta forma, las clases que necesitan los archivos de un paquete, solo tienen que importar el barrel de ese paquete y ya se tiene acceso a todos los archivos que contiene. la ventaja es que si en un futuro se le añaden nuevos archivos al paquete, no será necesario volver a tocar los `import` de las clases usuarias.



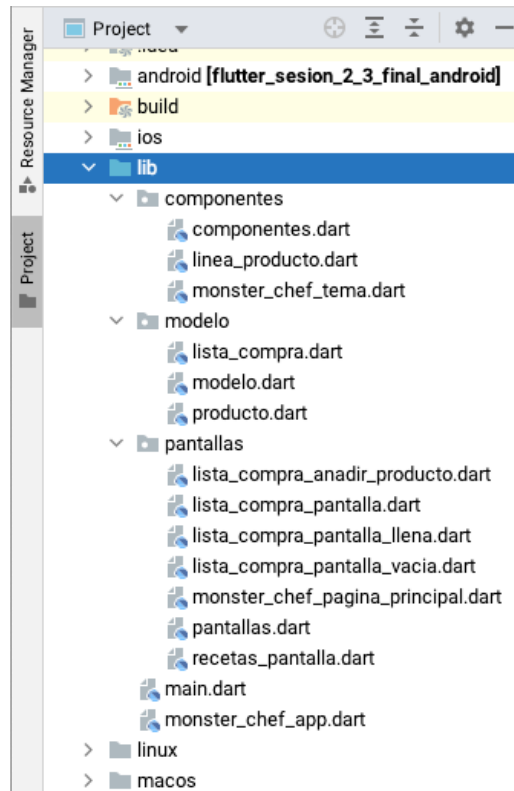
Ejemplo de **barrel**. Archivo `modelo.dart` con `export` a todos los archivos del paquete `modelo`

Nota: Lo normal es que el barrel tenga el mismo nombre que el paquete al que pertenece, pero no es obligatorio.

En cualquier archivo donde se necesite alguna de las clases del paquete `modelo` solo hay que añadir un `import` al barrel `modelo.dart` mediante la siguiente línea de código:

```
import 'package:flutter_sesion_2_3_final/modelo/modelo.dart';
```

Siguiendo el mismo procedimiento añadiremos al proyecto los otros dos paquetes y los colocaremos dentro las clases correspondientes. El resultado debería ser similar a este:



Estructura de paquetes del proyecto.

2 - Servidor de recetas

Para esta práctica, vamos a utilizar las recetas contenidas en el servidor **The Meal DB**, una base de datos de código abierto que contiene gran cantidad de recetas de todo el mundo.

Lo mas interesante de TheMealDB es que ofrece una API JSON gratuita para cualquiera que quiera usarla durante el desarrollo de una aplicación móvil para su puesta a punto, o para uso educativo.

El servicio REST TheMealDB

TheMealDB es un servicio REST.

Recuerda: Un servicio **REST** (REpresentational State Transfer), es una API que incluye órdenes HTTP como por ejemplo: GET, POST, PUT y DELETE para indicar el tipo de operación que vamos a realizar sobre los recursos que nos ofrece.

En una API REST la idea de "servicio" como tal desaparece. Lo que tenemos son recursos, accesibles mediante identificadores (URIs). Sobre esos recursos podemos realizar acciones, generalmente diferenciadas a través de verbos HTTP distintos.

Por ejemplo, en un servicio REST que albergue una colección de cervezas, podría haber un recurso llamado cervezas al que se accedería añadiendo a continuación de la dirección del servidor la cadena: `/cervezas`. Si se llama a la URI así formada se obtendría una representación de ese recurso.

Para acceder al servicio REST proporcionado por ThemealDB se usa la URI:

```
www.themealdb.com/api/json/v1/1/
```

La clave `1` se usa para acceder a la API gratuita.

Así por ejemplo, para filtrar todas las recetas de la categoría mariscos se usa la URI:

```
www.themealdb.com/api/json/v1/1/filter.php?c=Seafood
```

y para filtrar todas las recetas que pertenezcan a la cocina canadiense se usa la URI:

```
www.themealdb.com/api/json/v1/1/filter.php?a=Canadian
```

Formato JSON de TheMealDB

El servicio REST TheMealDB proporciona los resultados en formato JSON.

Recuerda: El formato JSON se usa muchísimo para serializar e intercambiar información dado que es muy sencillo de leer e interpretar para humanos y muy sencillo de escribir y leer en código.

Se basa en dos estructuras: mapa y lista, que se implementan en casi todos los lenguajes de programación, por tanto su lectura y escritura es simple ya que lo que se lee se puede volcar casi directamente a estructuras de datos en memoria.

La explicación del formato JSON va más allá de esta asignatura, pero puede encontrarse una buena descripción aquí: <https://www.json.org/json-es.html>

La respuesta que ofrece TheMealDB está en formato JSON con esta estructura:

```
{
  "meals" : [
    {
      "strMeal" : "Ayam Percik",
      "strMealThumb" : "https://www.themealdb.com/images/media/meals/020z181619788503.jpg",
      "idMeal" : "53050"
    },
    {
      "strMeal" : "Brown Stew Chicken",
      "strMealThumb" : "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg",
      "idMeal" : "52940"
    }
  ]
}
```

Veamos lo que significa cada elemento:

- Las llaves más externas indican que en la respuesta hay un único objeto, en este caso una lista de recetas, que está formado por un par de valores separados por `:`, la cadena `"meals"` que es la clave y una lista que se indica por los corchetes `[ ]`.
- En la lista hay dos objetos, lo que se indica mediante los dos pares de llaves que encierran a cada receta. Los distintos objetos de la lista se separan entre sí mediante comas `,`.
- Las propiedades de cada objeto están definidas mediante pares llave-valor, separado por `:`, así por ejemplo, en el primer objeto hay una propiedad `"strMeal"` cuyo valor es `"Ayam Percik"`.
- Cada objeto proporcionado por TheMealDB tiene tres propiedades:
 - `"strMeal"` que representa el nombre de la receta.
 - `"strMealThumb"` que contiene la URI de una imagen de esa receta terminada.
 - `"idMeal"` que es el código que identifica de forma unívoca a cada receta.

Clases del modelo

Como hemos visto en la sección anterior, TheMealDB proporciona un objeto para codificar cada receta y para cada una se tienen tres propiedades. Para representar esos objetos en la aplicación añadiremos al paquete `modelo` una nueva clase llamada `RecetaBasica` con el siguiente código:

```
class RecetaBasica {
    String id;
    String nombre;
    String urlImagen;

    RecetaBasica({
        required this.id,
        required this.nombre,
        required this.urlImagen,
    });
}
```

Veamos algunas consideraciones sobre esa clase:

- Se ha añadido una propiedad por cada propiedad de las recetas que proporciona TheMealDB. En este caso hay tres propiedades.
- Se han cambiado los nombres de las propiedades ya que los nombres de la respuesta en formato JSON son algo confusos. Se ha hecho la siguiente conversión:
 - `idMeal` se ha cambiado por `id`
 - `strMeal` se ha cambiado por `nombre`
 - `strMealThumb` se ha cambiado por `urlImagen`

Recuerda: Siempre que añadas una clase a un `package` debes añadir también su `export` al barrel de ese `package`

Decodificar recetas en formato JSON

Como hemos visto, las propiedades de cada receta en la respuesta JSON se dan en forma de grupo de pares llave-valor separados por `:` eso en Dart se implementa como un mapa. Las llaves en JSON son siempre cadenas de caracteres, mientras que los valores pueden ser listas o cadenas de caracteres, así que, cuando decodifiquemos la respuesta JSON, Dart nos dará las propiedades de cada objeto en forma de mapa de tipo `Map<String, dynamic>`

Nota: En este ejemplo, TheMealDB nos da todos los valores de propiedad también en formato `String`, pero no siempre es así, por eso el decodificador de JSON que incorpora Dart asigna tipo `dynamic` para los valores.

Para simplificar la construcción de instancias de `RecetaBasica` a partir de `Map<String, dynamic>` y la traducción de los nombre de llave a su correspondiente nombre de propiedad, añadiremos el siguiente método al final de la clase `RecetaBasica`

```
factory RecetaBasica.fromJson(Map<String, dynamic> json) {  
  return RecetaBasica(  
    id: json['idMeal'],  
    nombre: json['strMeal'],  
    urlImagen: json['strMealThumb'],  
  );  
}
```

Veamos algunos aspectos interesantes del método `fromJson` :

- Es un método `factory` ¿Recuerdas cuáles eran las ventajas de esta clase de métodos? ¿Por qué crees que se usa aquí?
- Tiene como parámetro un mapa, el que corresponde con alguno de los objetos JSON representando a una receta que proporciona el servicio REST TheMealDB. El mapa tiene tipo `Map<String, dynamic>` para adaptarse al caso más general.
- Con los distintos valores construye una nueva instancia de la clase `RecetaBasica` Para ello realiza la conversión de cada llave del mapa a su correspondiente propiedad de la clase.

3 - Leer las recetas de forma asíncrona

¿Qué son y cómo se usan los `Future` ?

En ocasiones, no se puede garantizar que una operación de lectura se complete en un tiempo breve. Esto ocurre, por ejemplo, cuando se leen datos de un servicio externo. En estos casos, podría ser que la velocidad de la red, la carga de trabajo del servidor, etc. hicieran que el tiempo de lectura sea elevado.

Si estas operaciones de lectura cuyo tiempo no está garantizado que va a ser breve se ejecutan en la hebra principal, bloquean la aplicación y hasta que no se terminen, la interfaz no va a responder a nuevos eventos.

Para poder realizar esas lecturas sin bloquear la aplicación se usa la clase `Future`. La Forma en que se usa en Dart es:

```
Future<TipoQueSeDevuelve> funcionQueGestionaElFuture() async {
  final valorQueSeLee = await funcionQueRealmenteHaceLaLectura();
  final TipoQueSeDevuelve valorQueSeDevuelve = procesar(valorQueSeLee);
  return valorQueSeDevuelve;
}

Future<TipoQueSeLee> funcionQueRealmenteHaceLaLectura() async {
  TipoQueSeLee valorLeido;
  // Realizar la lectura en valorLeido
  return valorLeido;
}
```

Algunas consideraciones a tener en cuenta:

- Lo normal es que las situaciones en las que se realice una lectura de un servicio externo se gestionen mediante dos funciones:
 - La función que **gestiona** la recopilación de información. Construye los datos que necesita la aplicación a partir de los datos que proporciona el servicio. Esta función realizará llamadas a la función de lectura.
 - La función que **realiza la lectura**. Es la que conecta con el servicio, realiza la petición de información y recoge los resultados.
- `Future` es una clase que encapsula a un valor (no necesariamente atómico, puede ser una colección) devuelto por una función que consume mucho tiempo.
- Cualquier función etiquetada con la palabra reservada `async` antes del cuerpo de la función, **debe devolver un `Future`**. La palabra `async` le dice a Dart que es una función asíncrona, y que su resultado no va a estar disponible inmediatamente. Dart la ejecuta en otra hebra distinta para no bloquear la hebra principal.
- Dentro de esa función asíncrona hay una o varias llamadas a funciones costosas en tiempo, que se marcan con la palabra reservada `await`. Cuando Dart ve la palabra `await` interrumpe la ejecución de la función hasta que se complete la llamada a la función costosa.

- Las funciones costosas en tiempo deben devolver también un `Future`, no necesariamente del mismo tipo. Es decir, el tipo leído y el tipo devuelto no tienen por qué ser iguales. En el caso de lecturas, es frecuente que se lean cadenas de caracteres, y en función de ellas se construyan objetos y se devuelvan.

Importante: No es obligatorio tener separada la función que realiza realmente la lectura, su código podría estar directamente en la función que gestiona el `Future`, pero de esta forma si se quiere cambiar el servicio sólo hay que cambiar la función de lectura.

Leer las recetas usando `Future`

Para poder usar las órdenes HTTP como por ejemplo GET y acceder al servicio proporcionado por TheMealDB, debemos añadir una nueva dependencia al proyecto:

```
http: ^0.13.5
```

Una vez añadida y reconstruidas las dependencias del proyecto, podremos escribir una nueva clase en el paquete `modelo` para gestionar el acceso al servicio de recetas.

Añadiremos un nuevo archivo al paquete `modelo` llamado `servicio_recetas_http.dart` y dentro escribiremos la clase `ServicioRecetasHttp`:

```
import 'package:http/http.dart';
import 'dart:io';

class ServicioRecetasHttp {
  Future<String> _cargarDesdeHttp(String url) async {
    final urlComprobada = Uri.parse(url);
    final respuesta = await get(urlComprobada);
    if (respuesta.statusCode == HttpStatus.ok) {
      return respuesta.body;
    } else {
      return '';
    }
  }
}
```

Veamos algunas consideraciones sobre el método `_cargarDesdeHttp`:

- El método `_cargarDesdeHttp` es quien realmente conecta con el servidor, envía la petición y recibe los resultados.
- Lo que se recibe desde el servicio REST es una respuesta en formato JSON, por tanto el tipo leído es una cadena de caracteres. Posteriormente habrá que procesar ese `String` para obtener las recetas a partir de él. Un poco más abajo escribiremos la función para gestionar los datos obtenidos del servicio.
- El método `_cargarDesdeHttp` es asíncrono ya que contiene operaciones que podrían tardar. Para ello se escribe la palabra reservada `async` delante del cuerpo.

- Como la operación de lectura podría tardar por problemas en la red, el servidor, etc. el resultado devuelto por el método no es de tipo `String` sino que es de tipo: `Future<String>`
- Recibe como parámetro una url con la petición que se necesite. Por ejemplo, para obtener las recetas que tengan como ingrediente al pollo, desde el servidor REST TheMealDB usaremos la url:

```
https://www.themealdb.com/api/json/v1/1/filter.php?i=chicken
```

- Antes de usar la url con el método `get` del paquete `http` es conveniente analizarla para comprobar que corresponde con una url válida (bien formateada). Para ello se usa la línea:

```
final urlComprobada = Uri.parse(url);
```

- A continuación se muestra la llamada que realiza la petición al servidor para obtener los resultados. Como no se sabe el tiempo que va a tardar se precede de `await`, lo que bloquea la hebra hasta que se completa la llamada.

```
final respuesta = await get(urlComprobada);
```

- Antes de devolver el valor, se debe comprobar el resultado de la petición al servidor, si `respuesta.statusCode` es igual a `HttpStatus.ok` (que equivale al código 200), entonces se devuelven los datos leídos que se encuentran en `respuesta.body` y si no, la cadena vacía `''`

Con la ayuda del método privado `_cargarDesdeHttp`, que obtiene del servicio REST una cadena de caracteres con las recetas, construiremos el método `getRecetas()` que devuelve una lista con las recetas leídas. En la clase `ServicioRecetasHttp` añadiremos el método:

```
Future<List<RecetaBasica>> getRecetas() async {
  final cadenaLeidaServidor = await
    _cargarDesdeHttp("https://www.themealdb.com/api/json/v1/1/filter.php?i=chicken");
  final Map<String, dynamic> json = jsonDecode(cadenaLeidaServidor);
  if (json['meals'] != null) {
    final recetas = <RecetaBasica>[];
    for (var receta in json['meals']) {
      recetas.add(RecetaBasica.fromJson(receta));
    }
    return recetas;
  } else {
    return [];
  }
}
```

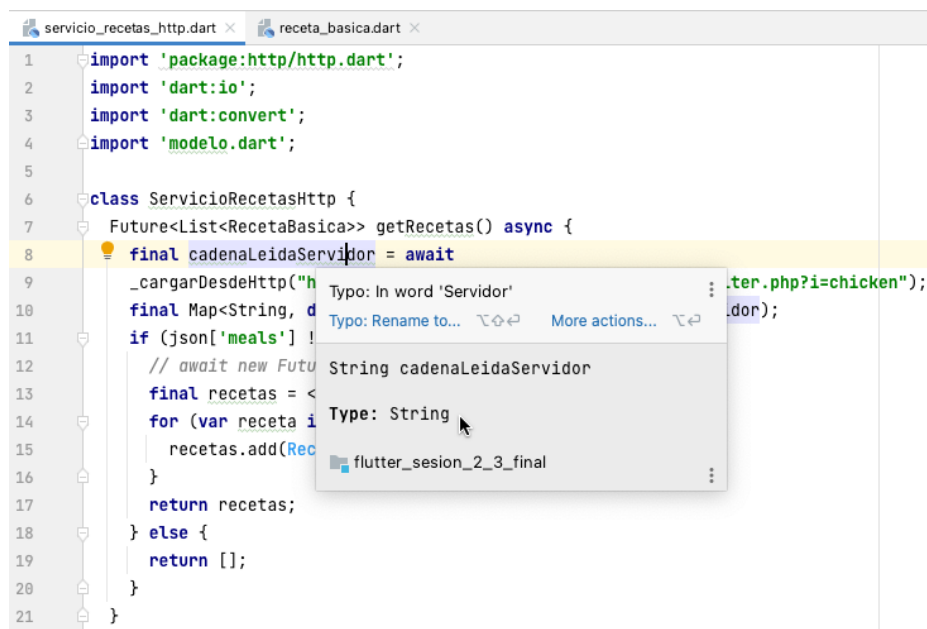
con los correspondientes `import`:

```
import 'dart:convert';
import 'modelo.dart';
```

Algunas consideraciones interesantes sobre el nuevo método:

- El valor devuelto por el método `getRecetas` es una lista de recetas: `List<RecetaBasica>`. Como puede tardar en completarse se marca como `async` para que Dart lo ejecute en una hebra aparte y no se bloquee la hebra principal, y como se ha marcado como `async` el resultado devuelto se envuelve dentro de una clase `Future`
- Dentro de `getRecetas` se hace uso del método `_cargarDesdeHttp` que como está marcado como `async` debe invocarse con la palabra reservada `await`. La palabra reservada `await` bloquea la ejecución de la función hasta que se complete la llamada a `_cargarDesdeHttp`
- El tipo que Dart infiere para la variable `cadenaLeidaServidor` es `String` a pesar de que el valor devuelto por `_cargarDesdeHttp` es `Future<String>`. Esto es así ya que cuando se devuelve el valor que se almacena en `cadenaLeidaServidor` el método `_cargarDesdeHttp` ha terminado por completo (recuerda que se invoca con `await`), por lo que ya se tiene un `String`

Se puede comprobar fácilmente si se coloca el puntero sobre `cadenaLeidaServidor` en Android Studio para que muestre el tipo inferido.



El tipo inferido para `cadenaLeidaServidor` es `String`.

- Después de la lectura, hay que procesar la cadena de caracteres `cadenaLeidaServidor` que contiene la respuesta en formato JSON, para extraer de ella las distintas recetas que contiene. Recordemos que en JSON se usan mapas y listas.
- Se usa el método `jsonDecode` que es proporcionado por el paquete: `dart:convert` por tanto hay que importarlo.
- `jsonDecode` devuelve un mapa. La llave siempre es de tipo `String` pero los valores pueden ser listas o cadenas de caracteres, por eso se usa como tipo para el valor que devuelve ese método:

`Map<String, dynamic>`

Como sabemos, el tipo `dynamic` es peligroso, pero en este caso se puede gestionar bien ya que para cada llave sabemos exactamente el tipo del valor asociado (porque conocemos el formato concreto de esta respuesta en formato JSON), por lo que podremos convertirlo sin problemas.

- Si la primera llave no es `meals` entonces es que se ha hecho una petición al servicio incorrecta y se devuelve una lista de recetas vacía.
- En caso contrario el valor asociado a la llave `meals` es una lista con todas las recetas. Se recorren todas las recetas y se delega en el constructor factoría de la clase `RecetaBasica` llamado `fromJson` que se vio en la sección anterior, para realizar la decodificación.
- Una vez terminado de analizar la respuesta del servicio se devuelve la lista de recetas.

4 - Construir la interfaz de forma asíncrona

Cuando una aplicación móvil está esperando a que se complete una tarea costosa en tiempo, debería indicarlo a su usuario de alguna forma, por ejemplo, con un **loading spinner**, y cuando la tarea termina, debería mostrarse la interfaz con los resultados.



Ejemplo de loading spinner.

Para simplificar este proceso, Flutter tiene la vista `FutureBuilder` a la que se le asocia un `Future` y permite mostrar una vista u otra, en función del estado del `Future`. Así por ejemplo, si la tarea costosa encapsulada en el future no se ha completado, se puede mostrar un **loading spinner** para indicar que la aplicación sigue trabajando, y si se ha completado, mostrar los resultados obtenidos, que en esta aplicación son los datos de recetas leídos.

Para ello, en la clase `RecetasPantalla` añadiremos la propiedad:

```
final servicioRecetas = ServicioRecetasHttp();
```

y a continuación buscaremos las líneas:

```
return const Center(  
  child: Text('Galería de recetas 🍷'),  
);
```

y las sustituiremos por:

```
return FutureBuilder(
  future: servicioRecetas.getRecetas(),
  builder: (context, AsyncSnapshot snapshot) {
    if (snapshot.connectionState == ConnectionState.done) {
      if (snapshot.hasError) {
        return const Center(
          child: Text('No se puede conectar con el servidor'),
        );
      } else {
        return Center(
          child: Text('Galería de recetas 🍷')
        );
      }
    } else {
      return const Center(child: CircularProgressIndicator(),);
    }
  },
);
```

Algunos aspectos interesantes del código añadido:

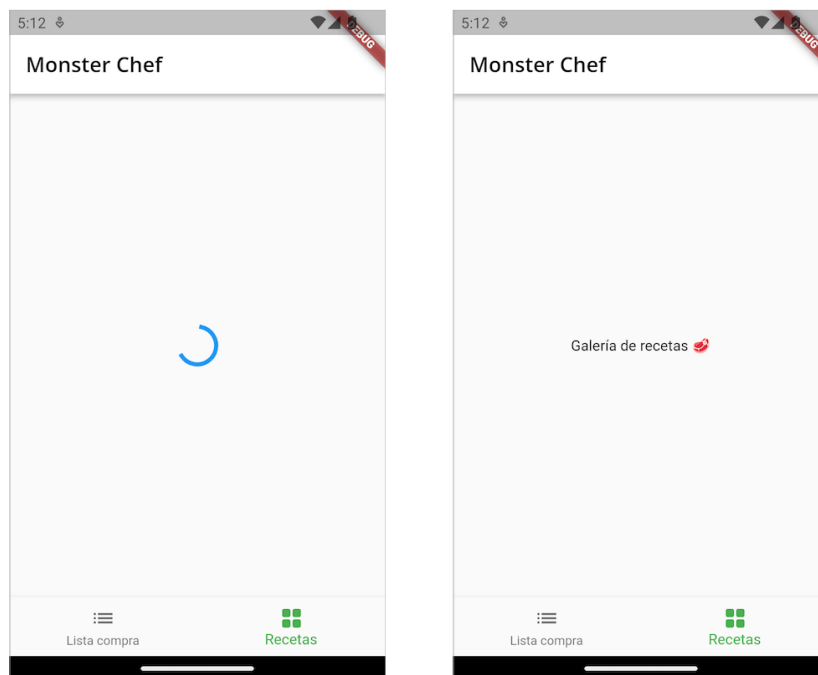
- `FutureBuilder` necesita dos propiedades:
 - `future`: donde se indica el future al que hay que esperar. El que contiene la tarea costosa en tiempo. En este caso el devuelto por el método `getRecetas()` de la clase `servicioRecetas`.
 - `builder`: que indica cómo construir la interfaz en las distintas situaciones: esperando y completado.

La función anónima asociada al `builder` recibe como parámetros un contexto y un `AsyncSnapshot` asociado al tipo devuelto. Es esta clase la que permite al `builder` interrogar al future sobre su estado. Para ello se usa la propiedad `connectionState` de `AsyncSnapshot`.

Si el future no se ha completado, se devuelve como vista un **loading spinner** mediante la clase de Flutter `CircularProgressIndicator` y cuando se completa, se devuelve la misma vista que se tenía antes de incluir la lectura del repositorio.

Nota: En próximas secciones se completará para que muestre una rejilla con las recetas leídas.

Si se hace hot reload y se selecciona la pestaña de **Recetas** se muestran las pantallas:



Pantalla de recetas cuando se no se ha completado la carga de las recetas (izquierda) y cuando se ha completado (derecha).

Durante el tiempo que dura la carga (normalmente muy breve) se muestra el loading spinner y cuando termina, aparece la pantalla con el texto **Galería de recetas** 🍷 Si hubiera algún error, se obtendría otra pantalla con el texto **No se puede conectar con el servidor**

Nota: Este último caso lo puedes probar seleccionando el modo avión en el emulador de Android directamente en los ajustes (settings) y a continuación saliendo de la pestaña de recetas y volviendo a entrar.

5 - Miniatura para mostrar las recetas

Para mostrar las recetas usaremos el **patrón galería**.

Recuerda: El patrón de interacción: galería, es un patrón de **navegación primaria persistente**. Muestra contenido altamente visual organizado en una cuadrícula.

En este caso, cada elemento de la cuadrícula, es una receta, para la que se mostrará su foto en miniatura y su nombre.



Para simplificar la construcción de la miniatura de cada receta, escribiremos una clase en el paquete `componentes` llamada: `MiniaturaReceta`

```
class MiniaturaReceta extends StatelessWidget {
  final RecetaBasica receta;
  const MiniaturaReceta({Key? key, required this.receta}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(6.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(
            height: 100,
            width: double.infinity,
            child: ClipRRect(
              borderRadius: BorderRadius.circular(12),
              child: Image.network(
                receta.urlImagen,
                fit: BoxFit.cover,
              ),
            ),
          ),
          const SizedBox(height: 4,),
          Text(
            receta.nombre,
            maxLines: 1,
            style: Theme.of(context).textTheme.bodyMedium,
          ),
        ],
      ),
    );
  }
}
```

Veamos algunas consideraciones sobre esta clase:

- No hay que gestionar ningún estado mutable, por tanto la clase `MiniaturaReceta` es una subclase de `StatelessWidget`
- Recibe como parámetro una receta que es un objeto de la clase `RecetaBasica`. Es la receta para la que se construye la miniatura.
- Se usa la clase `Padding` que permite establecer un espacio alrededor de su vista hija. Ese espacio se ha fijado en 6 dp.
- Como vista hija de la clase `Padding` se ha elegido un objeto de la clase `Column` que como sabemos permite organizar una lista de vistas hijas en modo columna.
- Dentro de la clase `Column` se usa la propiedad `crossAxisAlignment` para indicar que los elementos se alineen a la izquierda. Para eso se usa el valor `CrossAxisAlignment.start`

- En la columna hay tres vistas hijas:

- Un `SizeBox` que permite establecer una altura fija para todas las imágenes en miniatura de la rejilla. En este caso se ha fijado a 100 dp. La anchura de las miniaturas no es fija, sino que depende de cuántas haya en la pantalla, por eso el valor de la propiedad `width:` es `double.infinity` que en la práctica equivale a decir que se ocupe todo el espacio disponible.

Dentro del `SizeBox` hay una vista de tipo `ClipRect` que permite recortar los bordes en la vista hija con forma redondeada. Hay que pasarle un radio. En este caso se ha usado un borde circular con 12 dp de radio.

Dentro del `ClipRect` está la imagen que muestra la receta terminada. Se ha usado la vista de Flutter `Image` que está diseñada para mostrar imágenes en la interfaz de usuario. Dado que la imagen no se tiene en la memoria del dispositivo, sino que se tiene su url, se usa el constructor con nombre `network` de la clase `Image`. Este constructor, como su nombre indica, construye una vista `Image` y la rellena con una imagen descargada de Internet.

Para que todas las imágenes en todas las miniaturas de las recetas se dibujen del mismo tamaño, se usa `fit: BoxFit.cover`, De esa forma las imágenes se escalarán sin perder las proporciones para ocupar todo el espacio asignado a la miniatura.

- Una vista `SizeBox` para dejar un espacio entre la imagen y el nombre de cada receta.
- Una vista `Text` para mostrar el nombre de la receta. Como estilo se usa el estilo de texto: `bodyMedium` del tema de la aplicación, al que se accede a través del contexto.

6 - Usando el patrón galería

Para usar el patrón de interacción llamado galería se usa la vista de Flutter `GridView`. Una vista `GridView` es una vista de Flutter especializada en mostrar un array 2D de vistas que puede desplazarse (se puede hacer scroll). Las vistas hijas en un `GridView` se organizan en filas y columnas.

Para averiguar el espaciado y el número de columnas, `GridView` delega en un objeto de la clase `SliverGridDelegateWithFixedCrossAxisCount` que calcula esos valores en función del número de columnas que se le pasa como argumento.

Para simplificar la gestión de la galería, añadiremos otra clase al paquete `componentes` llamada `GridRecetas` donde encapsularemos el uso del `GridView` :

```
class GridRecetas extends StatelessWidget {
  final List<RecetaBasica> recetas;
  GridRecetas({Key? key, required this.recetas}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return GridView.builder(
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2,
        mainAxisExtent: 135,
      ),
      itemBuilder: (context, index) {
        return MiniaturaReceta(receta: recetas[index]);
      },
      itemCount: recetas.length,
    );
  }
}
```

Veamos con detalle esta clase:

- No hay que gestionar ningún estado mutable, por tanto la clase `GridRecetas` es una subclase de `StatelessWidget`
- Recibe como parámetro una lista de recetas `List<RecetaBasica>` que es la lista de recetas que debe mostrar como una galería.
- En el método `build` se devuelve una vista de la clase `GridView` que se construye mediante su constructor con nombre `builder` . Este constructor requiere tres propiedades:
 - `itemCount`: que indica a la clase `GridView` cuantos elementos debe representar como una cuadrícula. En este caso tantos como recetas haya, por eso se le da el valor `recetas.length`
 - `itemBuilder`: al que se le pasa la función anónima que construye la vista que representa a cada elemento de la cuadrícula. En este caso la función anónima lo que hace es devolver una instancia de la clase `MiniaturaReceta`

La función anónima que se pasa a `itemBuilder` se ejecuta una vez por cada receta que se muestra.

Siempre tiene dos parámetros: `context` de tipo `BuildContext` e `index` de tipo `int` , donde `context` es el contexto en el que se inserta la vista (proporciona su apariencia visual, etc.) e `index` indica el índice de la receta concreta que se está construyendo.

- `gridDelegate`: Hay que especificar un `gridDelegate` . En este caso se usa la vista predefinida `SliverGridDelegateWithFixedCrossAxisCount` que calcula el espaciado y el número de columnas a partir del valor que se le pasa en la propiedad `crossAxisCount` . En este caso `2` . La propiedad `mainAxisExtent`: indica el máximo espacio vertical que ocupará cada celda en la rejilla.

Para usar la clase `GridRecetas` en la vista `PantallaRecetas` se abre el archivo: `pantalla_recetas.dart` y se buscan las líneas:

```
return const Center(  
  child: Text('Galería de recetas 🍲'),  
);
```

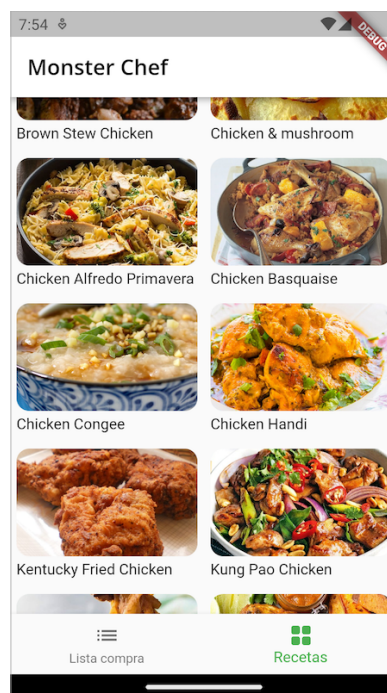
para sustituirlas por las líneas:

```
return Center(  
  child: GridRecetas(recetas: snapshot.data,),  
);
```

Veamos algunas consideraciones con respecto a este código:

- En la propiedad `data` de `snapshot` se encuentra el valor que se encapsula en el `Future`. En este caso, la lista de recetas de tipo `List<RecetaBasica>`. Cuando se construye la instancia de `GridRecetas` este es el valor que hay que pasar en el argumento `recetas`:

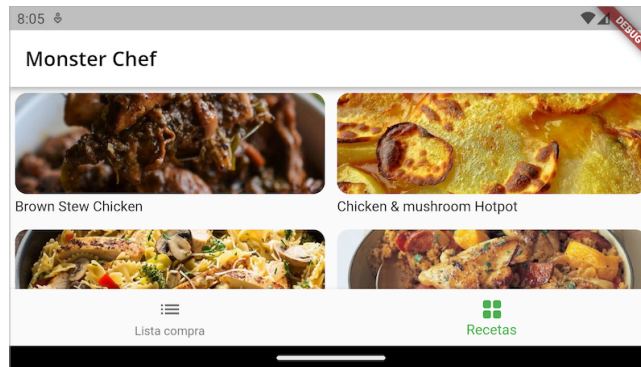
Si se hace hot reload y se selecciona la categoría `Recetas` al cabo de unos instantes podremos ver una pantalla similar a esta:



Pantalla de recetas terminada.

7 - Adaptar la interfaz a los cambios de orientación

La distribución (layout) de la pantalla de recetas es adecuada cuando se muestra en vertical, pero si se cambia de la orientación del dispositivo a la orientación horizontal, dos columnas son pocas y las miniaturas se muestran tan anchas que no se ven bien.

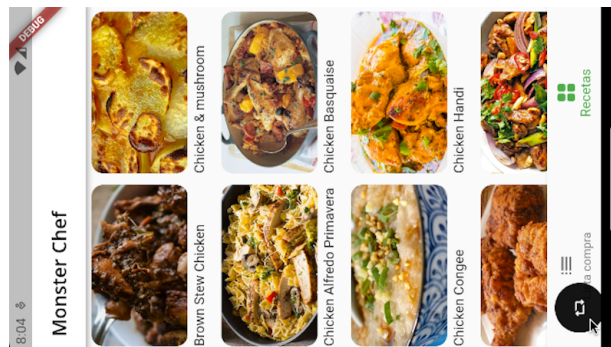


La distribución de las recetas con la pantalla en orientación horizontal no es adecuada. Las recetas no se ven bien.

Nota: En el emulador de Android, se puede conseguir orientación horizontal pulsando alguno de los dos botones con los iconos:



Es posible que en Android la orientación no se cambie realmente hasta que se pulse el botón de la doble flecha circular:



En estos casos sería conveniente tener un número variable de columnas que dependiese de la orientación del dispositivo. Para ello Flutter tiene la vista `OrientationBuilder` que permite saber qué orientación tiene el dispositivo y en función de ello adaptar el número de columnas.

Para usar `OrientationBuilder` buscaremos en la clase `GridRecetas` las líneas:

```
return GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    mainAxisExtent: 135,
  ),
  itemBuilder: (context, index) {
    return MiniaturaReceta(receta: recetas[index]);
  },
  itemCount: recetas.length,
);
```

y las sustituiremos por las líneas:

```
return OrientationBuilder(
  builder: (context, orientation) {
    return GridView.builder(
      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: orientation == Orientation.portrait ? 2 : 4,
        mainAxisExtent: 135,
      ),
      itemBuilder: (context, index) {
        return MiniaturaReceta(receta: recetas[index]);
      },
      itemCount: recetas.length,
    );
  },
);
```

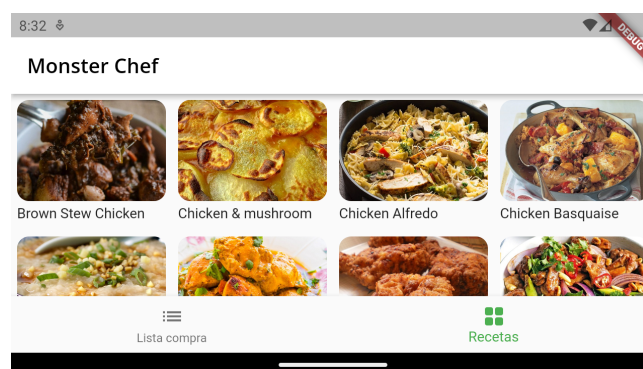
Veamos algunas consideraciones sobre este código:

- Se ha incluido el `GridView` dentro de una vista de la clase `OrientationBuilder`, que permite saber si el dispositivo está en posición vertical u horizontal y usar el número de columnas adecuado.

En el parámetro `orientation` se tiene siempre la orientación del dispositivo. Es una propiedad de tipo `Orientation` y puede tomar los valores `Orientation.portrait` (vertical) y `Orientation.landscape` (horizontal)

- En este código, se ha usado al especificar el número de columnas que se muestran en la rejilla. Si la orientación es vertical se usan dos columnas y si es horizontal 4. De esta forma las miniaturas se ven más proporcionadas y de tamaño similar en todas las orientaciones.

Si hacemos hot reload, y decimos al emulador que muestre la aplicación en orientación horizontal, el resultado es similar a este:



Rejilla de recetas en orientación horizontal corregida.

Como vemos, se parece mas a la orientación vertical y es mucho mas legible.

Mini ejercicios

1. ➡ Busca la línea:

```
_cargarDesdeHttp("https://www.themealdb.com/api/json/v1/1/filter.php?i=chicken");
```

dentro de la clase `ServicioRecetasHttp` y sustitúyela por la línea:

```
_cargarDesdeHttp("https://www.themealdb.com/api/json/v1/1/filter.php?i=chick");
```

Esa ruta no existe, así que el método `getRecetas()` de `ServicioRecetasHttp` devuelve una lista vacía. Cuando se construye el `GridView` a partir de ella no hay recetas que mostrar, por lo que la pantalla se muestra completamente en blanco.

¿Por qué crees que se devuelve una lista vacía y no se da un error de conexión?

¿Ese comportamiento es adecuado cuando se quiere una buena UX? ¿Cómo puede mejorarse la UX en esta situación?

2. ➡ Programa la solución que hayas propuesto para el problema que se describe en el ejercicio anterior.
3. ➡ ¿Qué habría que hacer para ver los detalles de una de las recetas pulsando en su miniatura? ¿Qué patrón de interacción usarías?
4. ➡ Para pedir a TheMealDB que devuelva los detalles de una receta, se usa la URI:

```
www.themealdb.com/api/json/v1/1/lookup.php?i=52772
```

donde `52772` es el `id` de la receta para la que se quiere consultar sus detalles .

Escribe esa URI en un navegador web, estudia la respuesta en formato JSON que te devuelve TheMealDB y construye una clase llamada `Receta` con su correspondiente método `fromJson` para parsear esa información.

5. ➡ Con la clase escrita en el ejercicio anterior, añade un método a la clase `ServicioRecetasHttp` para leer de forma asíncrona los detalles de una receta.
6. ➡ Usa todo lo anterior para añadir a la aplicación la capacidad de mostrar los detalles de una receta pulsando en su miniatura.